Implementation and evaluation of Hierarchical Semantic Wave Function Collapse for Procedural Tile Map Generation

Natalie Eriksson

naeriks@kth.se

KTH Royal Institute of Technology

Stockholm, Sweden

Anna Likhanova

annalik@kth.se

KTH Royal Institute of Technology

Stockholm, Sweden

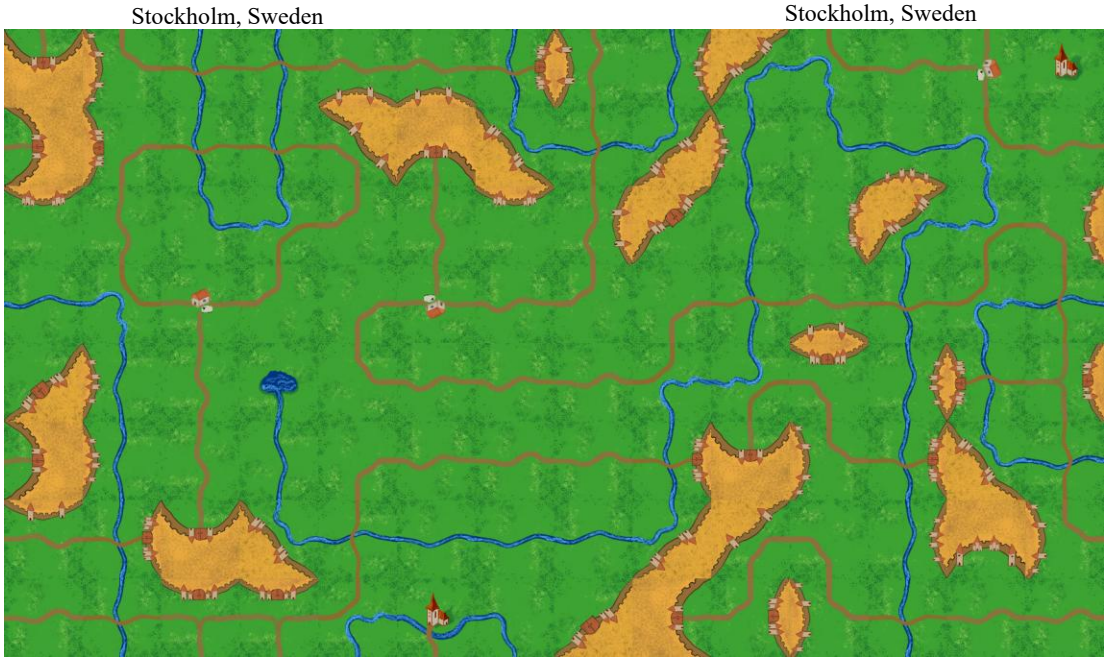

Figure 1: One output from the Procedural Content Generation program using our Hierarchical Semantic Wave Function Collapse created in this project

ABSTRACT

This project explores the implementation of the Hierarchical Semantic Wave Function Collapse algorithm for Procedural Content Generation. It gives an overview of related work in the field of PCG and WFC algorithms. This project looks at HSWFC in the context of tile map generation for board- and digital games, through a Python implementation. This project also touches upon the challenges in PCG and WFC: it raises questions about the controllability of the algorithm and suggests several avenues of evaluating it. The implementation in this project increased the level of control through meta-constraints and the addition of weighted edges.

1 INTRODUCTION

Procedural Content Generation (PCG) is an algorithm that produces content for applications such as games [1]. PCG can

be desirable for adding variety, surprise and replayability to games [1] as well as reducing the time and cost of game development [2]. Randomness is one of the important characteristics of PCG, and to create a good game through PCG the replacement of uniform randomness with directedness might serve as an important factor [3]. Carefully designing individual pieces of content that will be assembled through random content creation is one way of directing it, another way is through the assembling algorithm.

One approach for content generation that often has been overlooked is constraint solving, where Wave Function Collapse (WFC) is one constraint solving approach for PCG that can generate images in incremental steps [4]. WFC was created in 2016 by Maxim Gumin [5]. WFC grows its output in increments, and this can be visibly observable and is often captivating for humans to watch, particularly because we are not used to seeing generators working this way [4]. Due to WFC's constraint-based approach its output will more closely follow the wishes of the user [4] and it is also possible to combine with other generative models or support combinations of manual generation [5], however its execution time is not very predictable [4]. This unpredictable behavior of WFC remains of the main challenges in the algorithm's use for PCG. Mixed Initiative (MI) approaches have been suggested, some for example, utilize user-controlled "undo" mechanisms [6]. While MI adaptation of WFC is a step in the right direction, the experience shows that the burden of solving constraints is then largely passed onto the designers, leading to an increase in cognitive load [6]. Another way to allow users more control and decrease cognitive load is by implementing Hierarchical-Semantic WFC (HSWFC) [6, 8]. HSWFC introduces semantic representation of tiles through meta-tiles that can undergo intermediate collapses before collapsing to a terminal tile, which is a type of tile used by the generic WFC [6,7,8].

This project attempts to implement a HSWFC and look into the challenges of implementation and evaluation of the method.

This project explores how HSWFC can be used for creating 2D game maps for both digital and board games. This project utilizes tiles inspired by Carcassonne – a tile-laying board game which gameplay consists of developing an area around the city of Carcassonne [9]. Laying tiles to generate an area is the way that HSWFC algorithm works so the choice of tiles seemed natural and it also fits with our goal of exploring both digital games and board games.

2 RELATED WORK

The next subsections go through related work covering some challenges of PCG, some work done on WFC and then introduces HSWFC.

2.1 Challenges of PCG

There are some challenges with PCG. PCG often only generates one type of content that is for a specific game [2]. The generated content often ends up looking generic and lacking a sense of purpose and coherence. There is a challenge of controllability, where the desired type of control can vary depending on what you want to create. The interface does not provide the user with an easy way for control, some control can be encoded but that is not easily accessible for non-programming game developers. Mixed-initiatives PCG let users create and move things in the physical space, therefore providing some control of location.

The goal would be to create a generator that can create multiple types of content for multiple games with different functionality and aesthetics through easily changeable parameters, preferably generate an entire game with content that seems meaningful and thoroughly designed and offers some control to the developers [2].

2.2 Exploration of WFC

Karth and Smith explored Gumin's WFC algorithm by doing experiments of their surrogate of WFC to explore some questions [4]. Their findings suggest that WFC strengths are rather from its constraint-based approach rather than its use of entropy, in other words the strength is that WFC removes bad choices of tiles rather than that it picks the tile with the lowest entropy, which leads to few conflicts [4]. They also suggest that backtracking might be needed instead of only global restarting when finding conflicts like Gumin's WFC does.

Local backtracking has been added to WFC by others such as Oskar Stålberg [4].

2.3 Hierarchical Semantic WFC

Alaka introduced a version of WFC called Hierarchical Semantic WFC (HSWFC) in his master thesis [8], and he later on published two related articles about the topic together with Bidarra [6, 7]. His work tries to implement a mixed approach WFC that improves designers' ability to quickly sketch and draw to later let a generative system fill in the details [6]. To be able to create this, something called meta-tiles were introduced, which are an abstract tile that semantically represents a group of tiles as well as a graph that represents the hierarchy of the tiles along with their constraints. HSWFC is based on the notion that WFC ignorance of semantics blurs semantical concepts together which creates information-dense tiles in the output while also making it hard to find the designers initial semantic intent of choosing a tile since it is not stored anywhere. To represent the semantical concept a graph was chosen where the nodes are the semantical concepts, and the edges are the relationship between these concepts. The relationship that needs to be represented is a "consists of" relation and therefore the graph is acyclic, its difference from a tree is that it can have multiple parents. Therefore, a directed acyclic graph (DAG) is suitable to represent this type of relation. The further away from the root, the more detailed tiles are.

HSWFC works by initializing all cells on the grid to the meta-tile that is the root of the DAG [6]. There is also a set of adjacency constraints that might be specified amongst the tiles. Cell collapses to intermediate state of meta-tiles through choosing the direct children of the DAG node that represents the current tile. Therefore, a cell can collapse multiple times. The edges are weighted and will affect which tiles that get chosen. The collapse path is also stored in the output, allowing backtracking from the tile to its higher-level meta-tile with its semantical representation. For more information about HSWFC implementation see [6-8].

This project was influenced by Alaka's work on HSWFC, and the next section will go through the implementation of our project.

3 IMPLEMENTATION

This section will go through the implementation, starting with the tile creation, then implementation of a generic WFC, and finishing up with our HSWFC.

3.1 Tile creation

The first step involved creating a set of terminal tiles that were based on the tiles for the game of Carcassonne [9]. Each tile was drawn manually in Affinity Studio – free creative software for raster and vector graphics [10]. See figure 2 for an example of one of our tiles.

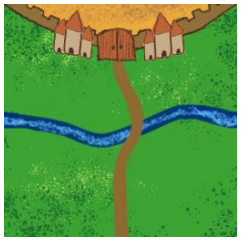

Figure 2: Created tile with ID 26

Each tile was drawn on a 256x256 pixel canvas and then exported as a .png in two resolutions – 256x256 and 128x128 pixels. We decided on these two resolutions since exporting tiles in smaller resolutions would lead to the loss of detail and visual fidelity.

3.2 Generic WFC

The algorithmic implementation started with implementing a WFC algorithm in Python [11] using pygame [12]. We used source code made by Uriel Garcilazo-Cruz [13]. This code used set of tiles with a corresponding file of metadata where each row described the id of the tile, the so-called sockets of the tiles and if the tile can be rotated or not. The id corresponded to the filename of the tile's .png. The sockets went through the tiles side starting with the right side and ending with the up side. The sockets contained information about how the tile looked and therefore how they could be matched to another tile. The last part of the metadata is a

binary number where 0 represents that the tile can rotate and 1 represents that the tile is fixed.

The sides of our tiles can consist of grass, road, castle or water, encoded as G R C or W. We split each side into three different parts, based on the design of our tile where one letter represented what that part of the tile consists of. The metadata for tile 26 (see figure 2) would therefore be 26 GWG GRG GWG CCC 0. Where 26 is the file name, the GWG standing for the grass, water, grass part of the right side of the tile etc. and the 0 representing that the tile can be rotated.

3.3 HSWFC

The WFC algorithm, with our tileset and it's corresponding metadata was then adapted for HSWFC by creating a weighted Directed Acyclic Graph (DAG) in accordance with [4], implementing a depropagation method used for backtracking and adapting the `collapse()` function.

A set of semantic meta-tiles was developed (figure 3). Every terminal tile was assessed and categorized accordingly. The categorized tiles were then added to a weighted DAG – adding weight to each edge further was an additional layer of control we have over the output of the algorithm. DAG was implemented using NetworkX [14]. Tile 26 (figure 2) represents meta tiles "Castle" and "Water" and by its transitive property it also represents tiles "Land" and "Root". It does not represent "Road" as tiles could only classify as such if they had no house or castle attribute.

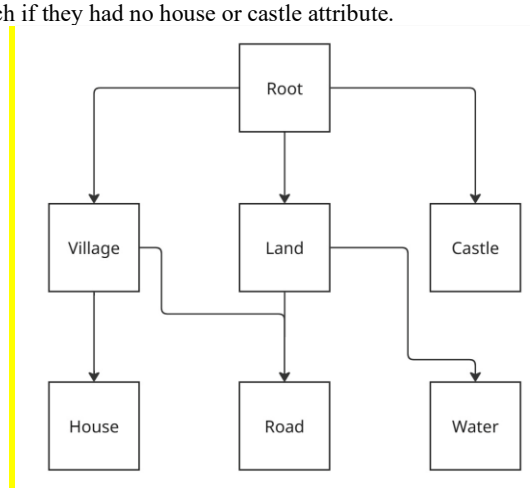

Figure 3: Semantic Meta Tiles

Alaka stated, that meta-tiles adjacency constraints could either be defined explicitly or be inferred from the meta-tree [8]. We decided to specify meta-tiles adjacency constraints explicitly: a "Castle" tile could not be directly adjacent (i.e. directly to the left, right, up or down) to a "House" tile. See the snippet of code below.

```
def metaConstraints(currentTile,
neighborTile):
    if dag.has_edge("house",
currentTile["ID"]) and dag.has_edge("castle",
neighborTile["ID"]):
        return False
    if dag.has_edge("castle",
currentTile["ID"]) and dag.has_edge("house",
neighborTile["ID"]):
        return False
    return True
```

This constraint was checked when updating the entropy of a tile, by checking if a tile is allowed to be adjacent to the current tile.

During the development we ran into an issue – the algorithm would stall when there were no valid potential tiles to choose from. To mitigate this, depropagation or "explosion" of tiles that did not have valid potential tiles and its neighboring collapsed tiles was implemented. Explosion simply marks the tile as uncollapsed and resets the potential terminal tiles to all tiles and does the same to all its collapsed neighbors. However, it still led to some cases where the algorithm got stuck, when the depropagation of the tile and its neighbors were not sufficient. Therefore, we implemented a stack that is filled with collapsed tiles, and when the current tile has no potential tiles, the last collapse tiled is also deprogagated along with the current tile and their neighbors.

As mentioned, the main `collapse()` function had undergone revision as well. It now takes the aforementioned weights into consideration when picking a tile, and not simply doing it randomly. If there are several tiles with the same weight, then it randomly picks between those tiles. It also now

call the `explode_neighbors()` method when the previous iteration of the method would stall. This ensures that the algorithm will not terminate wrongfully or get stuck, instead it will simply backtrack to the place where it has enough options again.

4 RESULT

The resulting algorithm produces tilemaps that follow the explicitly defined meta-tiles adjacency constraints. Using the weights in the DAG we are able to adjust output from the back end. The way the DAG is implemented allows us to control the frequency (via the weights) of both meta-tiles and individual terminal tiles. See figure 4, 5, and 6 for examples.

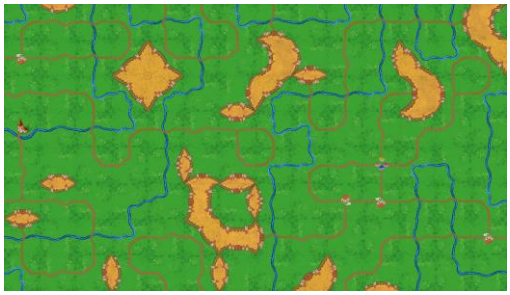

Figure 4: Output with the balanced weights

Figure 5: Output with the weights favoring the "village" meta-tiles

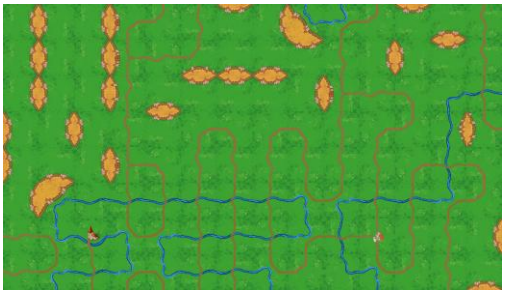

Figure 6: Output with the weights favoring terminal tile 11 of type "Castle"

5 EVALUATION

This section goes through possible evaluation for this algorithm.

5.1 Satisfying design constraints

One evaluation inspired by [15] would be to compare the WFC algorithm with the HSWFC algorithm implemented in this project based on set design constraints. The constraints would assess the level of controllability between the different algorithms, since if the algorithm can comply with the constraints that means it has a higher level of controllability than if it cannot. This test would simulate a situation of a game designer having constraints of what they want to design and how well the algorithms can provide that. The constraint could be that no house tile should be placed adjacent to a castle tile. Since there is some randomness in the algorithms they will be run several times, to more accurately see how well they follow the constraints. Therefore, each algorithm should be executed 10 times and create a 20 x 20 scene. Then the output should be evaluated whether it follows the constraints or not by checking if there is a house placed adjacent to a castle, if it is that will count as a failure. Each algorithm would get a score between 0 (never followed the constraints) to 10 (always followed the constraints), these results would then be compared between the algorithms to see which one followed the constraints most closely.

5.2 Computer generated vs human generated

Another evaluation that is also inspired by [15] is to compare the output from our algorithm with a human created map. We

would invite 5 game designers that each would manually piece together a 20x20 map. Then we would generate 5 computer generated maps through our program. We would then randomly mix those maps in a questionnaire seek for participants to answer it. We would inform the respondents that 5 maps are created by a human, in accordance with [15], and ask them to choose 5 levels which they think are created by a human. The desired result would be answers close to 50% thinking that it is human made and 50 % thinking that it is computer generated. That would mean that it is hard to distinguish between them, indicating that the result from our program generates content that feels as creative and nongeneric as something created by a human which is desirable [4]

5.3 Tile co-occurrence

As we were developing this project it became evident that some patterns emerge as the algorithm runs. Some tiles are more likely to be adjacent to others. One evaluation method could look into such patterns and see if the distribution of tiles in the resulting map corresponds to the weight encoded for each tile. This could be done via expressive range evaluation. Expressive range refers to the space of potential levels that the generator is capable of creating, including how biased it is towards creating particular kinds of content in that space [16, 17]. This would require definition of comparison metrics and iterating the algorithm a large number of times. The generated output would then be evaluated using those metrics as axes. The results would be plotted into a heatmap. The heatmap could reveal biases in the algorithm, and comparisons of the heatmaps across different sets of input parameters could show how controllable the algorithm is [16].

6 DISCUSSION AND FUTURE WORK

6.1 Discussion

This project looked into the implementation of the Hierarchical Semantic Wave Function Collapse algorithm for Procedural Content Generation. While some aspects of the algorithm were successfully implemented – such as the set of meta-tiles and depropagation method – which allowed the algorithm to generate output that corresponded to our expectations, it is clear to us that what we have implemented cannot be called a true HSWFC. First of all, though the algorithm respects the meta constraints we enforced, the way the code is written suggests that the tiles collapse directly to a terminal tile instead of a meta-tile, going against the HSWFC principles [6,8].

The divergence from the example algorithms can be explained by our choice of tile design. The script for generic WFC used as a starting point suggested three categories per socket, which Carcassonne style of tile fit pretty well. However, the way the terminal tiles turned out, allowed for little to no hierarchy: for example, “castle” did not consist of walls and floors as Alaka’s tiles of a house did [8] but rather of “castle” as the tiles were not separated into each element that would make up a castle. Similar fate struck the “house” tile. This then led to the simplification of the algorithm at hand and changing the way we handled hierarchy in our code.

However, the addition of a weighted DAG allowed us to control the output more than a generic WFC and the result our algorithm produces adheres to the set of adjacency rules we established. This is what makes this project different from a generic WFC – it follows a set of rules and allows for slightly more control over the output.

One potential use of the algorithm created in this project would be for developers of Carcassonne to use this algorithm to check how tiles of new expansions would fit into already existing tiles of Carcassonne.

The addition of weight adds a level of control, as previously discussed. Although weighting the tiles leads the algorithm to use certain tiles more often than others, leading to results that have specific patterns and that look more generic (as seen in the results), which is also one of PCG’s challenges [2]. Therefore, there seems to be a trade-off between control and a generic look. However, more investigation into this is needed before any conclusions are drawn. This could be approached by evaluating the human perception of generated content, as mentioned in section 5.2, and the actual tile co-occurrence as mentioned in section 5.3, to see if weight actually changes the number of patterns and the perception of it. Then those results would need to be

compared with the level of controllability which could be evaluated as mentioned in section 5.1.

One of the main challenges with PCG is the lack of control [2]. Based on how WFC works, the process of WFC is more visible than other PCG algorithms [4]. This should improve the level of understanding of the generation of content, which in a way improves controllability since it is harder to control something that is very abstract. Through this project it has been easy to check if the algorithm follows the rules and constraints that we have set, as well as how the different weights affect the outcomes, through the output it gives us. Although the control is made through code the display of the incremental creation improves understanding what the back-end code does and makes it easier to adjust and therefore control its output. As mentioned earlier, our project adds additional control to what generic WFC does, however more control could be added in the future, which is further discussed in the next subsection 6.2.

6.2 Future Work

Future work regarding this project would consist of either adopting the HSWFC algorithm to handle the set of tiles we developed or redesigning the tiles to be more in line with what previous research had used. Another path could be to do evaluation and comparison between our algorithm, Alaka's HSWFC algorithm [8] and WFC. Specifically comparing them using our tileset vs the tile set Alaka created [8] would be interesting, in order to explore if some algorithm works better on a certain types of tiles than others.

Another avenue that interests us is creating an interactive editor and evaluation of the process of using the algorithm. Creating an interactive editor would help increase the control of the generation, diminishing on of the great challenges of PCG and facilitating the access of control to non-programming designers [2]. Adding an editor also provides the opportunity to further evaluate other issues of PCG and WFC, such as the cognitive load of the designers using it which should decrease when using HSWFC [6, 8].

7 CONCLUSION

This project generated 2D game maps based on Carcassonne inspired tiles through an implementation of Hierarchical Semantic Wave Function Collapse. Control is one of the challenges of PCG. In this project the semantical constraints between meta-tiles and the weighted edges in the directed acyclic graph increases the level of control, which differentiated this project from the use of generic WFC. Evaluation of this project is needed before further conclusions are drawn.

8 CONTRIBUTIONS

The creation of tiles was done by Anna Likhanova. The implementation of WFC was done by Natalie Eriksson. The adaptation of HSWFC was done by both authors and both authors wrote the report.

REFERENCES

[1] Gillian Smith. 2015. An Analog History of Procedural Content Generation. 2015. . Retrieved May 17, 2026 from https://www.semanticscholar.org/paper/An-Analog-History-of-Procedural-Content-Generation-Smith/2400a5a51c8ff438f9e080f6c21735853916344e

[2] J. Togelius, A.J. Champandard, Pier Luca Lanzi, M. Mateas, A. Paiva, Mike Preuss, and K.O. Stanley. 2013. Procedural content generation: goals, challenges and actionable steps. Dagstuhl Follow-Ups 6, (January 2013), 61–75.

[3] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. 2013. Crowdsourcing the Aesthetics of Platform Games. IEEE Trans. Comput. Intell. AI Games 5, 3 (September 2013), 276–290. https://doi.org/10.1109/TCIAIG.2012.2231413

[4] Isaac Karth and Adam M. Smith. 2017. WaveFunctionCollapse is constraint solving in the wild. In Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG '17), August 14, 2017. Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/3102071.3110566

[5] Maxim Gumin. 2016. Wave Function Collapse Algorithm. Retrieved May 17, 2026 from https://github.com/mxgmn/WaveFunctionCollapse

[6] Shaad Alaka and Rafael Bidarra. 2024. Mixed-initiative generation of virtual worlds - a comparative study on the cognitive load of WFC and HSWFC. In Proceedings of the 19th International Conference on the Foundations of Digital Games (FDG '24), July 05, 2024. Association for Computing Machinery, New York, NY, USA, 1–8. https://doi.org/10.1145/3649921.3659852

[7] Shaad Alaka and Rafael Bidarra. 2023. Hierarchical Semantic Wave Function Collapse. In Proceedings of the 18th International Conference on the Foundations of Digital Games, April 12, 2023.

ACM, Lisbon Portugal, 1–10. https://doi.org/10.1145/3582437.3587209

[8] Shaad Alaka. 2023. Hierarchical Semantic Wave Function Collapse. Master's thesis. Delft University of Technology, Delft, NL. URL: http://resolver.tudelft.nl/uuid:e79264cc-1d0a-4151-9158-342a96d46764.

[9] Jean-François Gagné. 2014. Carcassonne: Base game rules. Hans im Glück Verlags-GmbH. Retrieved from https://cundco.de/media/06/fa/39/1715104334/base-game-2-1-rules.pdf

[10] Affinity | Professional Creative Software, Free for Everyone. Affinity. Retrieved May 17, 2026 from https://www.affinity.studio

[11] Python Software Foundation. Python Language Reference, version 3.12.1 Available at http://www.python.org

[12] Pygame. version 2.6.0 Available at https://github.com/pygame/pygame

[13] Uriel Garcilazo-Cruz. 2023. Wave Function Collapse. Retrieved May 17, 2026 from https://urielgarcilazo.com/tutorials/4_WaveFunctionCollapse.html

[14] Aric A. Hagberg, Daniel A. Schult and Pieter J. Swart, "Exploring network structure, dynamics, and function using NetworkX", in Proceedings of the 7th Python in Science Conference (SciPy2008), Gäel Varoquaux, Travis Vaught, and Jarrod Millman (Eds), (Pasadena, CA USA), pp. 11–15, Aug 2008

[15] Darui Cheng, Honglei Han, and Guangzheng Fei. 2020. Automatic Generation of Game Levels Based on Controllable Wave Function Collapse Algorithm. In Entertainment Computing – ICEC 2020, 2020. Springer International Publishing, Cham, 37–50. https://doi.org/10.1007/978-3-030-65736-9_3

[16] Noor Shaker, Gillian Smith, and Georgios N. Yannakakis. 2016. Evaluating content generators. In Procedural Content Generation in Games. Springer International Publishing, Cham, 215–224. https://doi.org/10.1007/978-3-319-42716-4_12

[17] Gillian Smith and Jim Whitehead. 2010. Analyzing the expressive range of a level generator. In Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames '10), June 18, 2010. Association for Computing Machinery, New York, NY, USA, 1–7. https://doi.org/10.1145/1814256.1814260